



ResQ Nepal

Project Report

Course Code :- CSE2006

Slot :- B14+D21

Registration Number :- 25BAI11110

Submitted By :- Khushi Singhi

Submitted To :- Prof.MUTHUSUNDARI

Github repository link :- <https://github.com/singhikhushi30-crypto/ResQ-Nepal-Khushi>

INDEX

S.NO	TITLE	PAGE NO.
1.	INTRODUCTION	3
2.	OBJECTIVES	3
3.	PROBLEM STATEMENT	3
4.	FUNCTIONAL REQUIREMENTS	4
5.	NON-FUNCTIONAL REQUIREMENTS	4
6.	SYSTEM ARCHITECTURE	4
7.	DESIGN DIAGRAMS	6
8.	DESIGN DECISIONS & RATIONALS	10
9.	IMPLEMENTATION	10
10.	SCREENSHOTS AND RESULTS	11
11.	TESTING	13
12.	CHALLENGES FACED	14
13.	LEARNING & KEY TAKEAWAYS	15
14.	FUTURE ENHANCEMENTS	15
15.	CONCLUSION	15
16.	REFERENCES	15

1. INTRODUCTION

Disaster response can be difficult because many emergencies can happen at the same time, while there are only limited rescue teams, equipment and shelter spaces available. While responding to an incident, things like how serious the incident is, which teams are available, what skills they have and how far they are from the location can be important.

ResQ Nepal is a Java-based disaster management and rescue coordination system that we made as an educational project. It is used to show how different things like victims, rescue requests, rescue teams, resources, shelters, missing persons, disasters and rescue operations can be managed in one application.

The main feature of the system is processing rescue requests based on priority. We use a Java PriorityQueue with a custom Comparator to arrange the requests according to their severity. When a request needs to be handled, the system checks which rescue teams are available and have the required skills. It also checks the distance to find a suitable team.

The application uses JavaFX with an MVC structure. FXML is used for creating the different screens, controllers handle the user actions, service classes handle the main logic, and DatabaseManager is used for connecting the application to the local SQLite database.

Through this project, we got to use concepts like OOP, abstraction, inheritance, custom exceptions, collections, JavaFX events, JDBC, SQL, exception handling and JUnit 5 testing.

This project is only made for educational purposes. The data used in it is fictional and it is not based on official Nepal statistics or an actual emergency-response system.

2. OBJECTIVES

1. Develop a Java-based system for managing disasters and coordinating rescue work.
2. Manage details like victims, rescue requests, teams, resources, shelters, missing persons and rescue operations.
3. Use PriorityQueue and a severity-based Comparator to give priority to rescue requests.
4. Find the nearest available rescue team that has the required specialization.
5. Check whether enough resources are available and whether shelters have space.
6. Keep records of missing persons and rescue operations.
7. Store the application data using SQLite and JDBC.
8. Use MVC architecture and keep the controllers and main business logic separate.
9. Use custom exceptions for handling errors and JUnit 5 for testing the system.

3. PROBLEM STATEMENT

During a disaster, many people can need help at the same time, but the number of rescue teams, equipment and shelter space is limited. Because of this, the more serious cases should be handled first instead of just following the order in which requests are received.

While choosing a rescue team, it is also important to check if the team is free, has the required skills and is near the incident location. The system also needs to keep track of the available equipment and space in shelters.

ResQ Nepal tries to manage these situations by giving priority to important rescue requests, finding a suitable team, checking resources and shelter space, and keeping records of the rescue work.

4. FUNCTIONAL REQUIREMENTS

Module	Main Functions
Dashboard	Show rescue-operation statistics, team availability and shelter capacity.
Victims & Requests	Register victims, create rescue requests and update request status.
Priority Processing	Order and process requests according to severity using PriorityQueue.
Rescue Teams	Manage members, location, availability and specialization.
Team Assignment	Select the nearest available team with the required specialization.
Resources	Maintain equipment inventory and allocate/release resources.
Shelters	Monitor capacity and admit evacuees without exceeding maximum capacity.
Missing Persons	Store missing-person records and update discovery status.
Rescue Operations	Track active and historical rescue operations.
Disasters / Simulation	Represent fictional educational disaster-response scenarios.

Overall Input–Output Flow

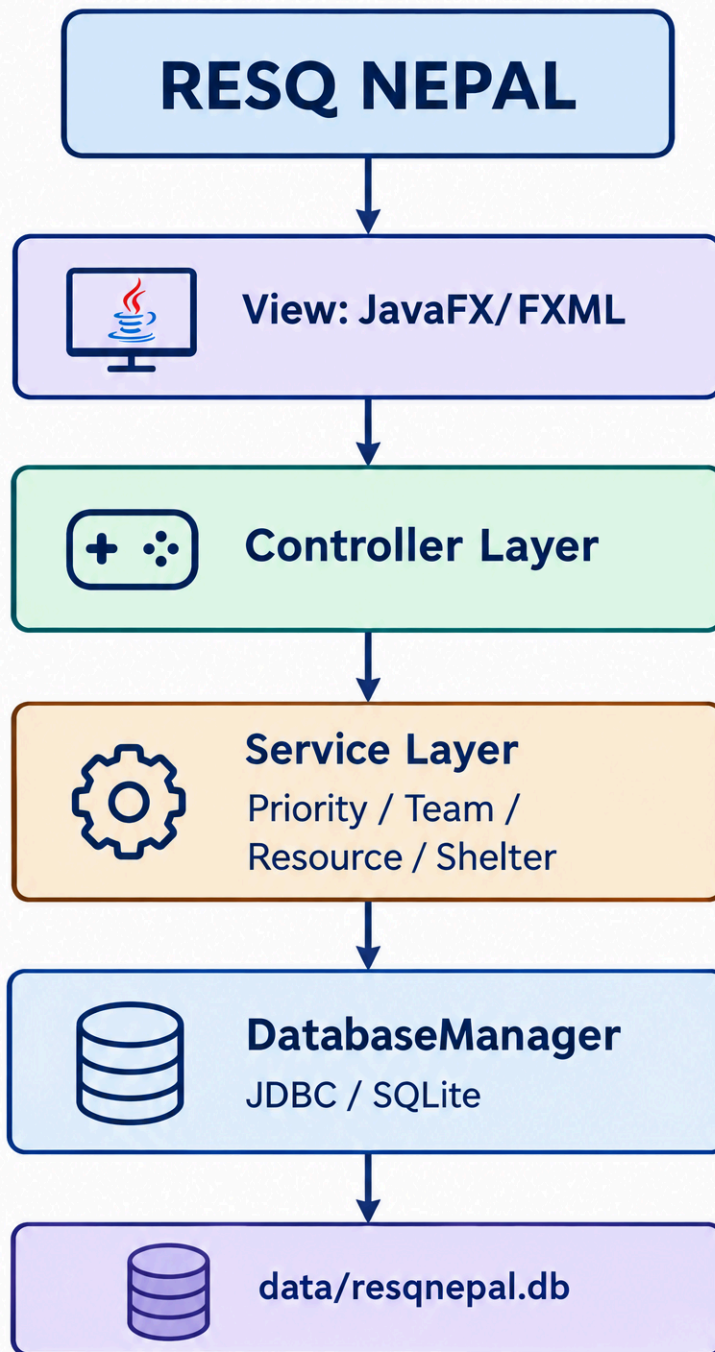
Victim / Incident Data → Rescue Request → Severity Priority → PriorityQueue → Suitable Team Search → Distance Check → Team Assignment → Resource Validation → Rescue Operation → Status Update

5. NON-FUNCTIONAL REQUIREMENTS

Requirement	Description
Usability	JavaFX screens should provide clear navigation and understandable controls.
Reliability	Valid operations should produce consistent results and preserve stored records.
Error Handling	Invalid requests and unavailable-team conditions should be handled through validation and custom exceptions.
Data Integrity	SQLite foreign-key enforcement should maintain relationships between related records.
Maintainability	Controller, model, service, repository and utility packages separate responsibilities.
Performance	PriorityQueue and SQLite provide efficient processing suitable for project-scale data.
Security	Parameterized SQL/PreparedStatement should be used for database values; the project is not production-grade security.
Scalability	The modular design should permit additional teams, scenarios, resources and integrations.
Testability	Core business logic can be tested independently with JUnit 5.

6. SYSTEM ARCHITECTURE

ResQ Nepal uses a JavaFX MVC architecture with a separate service layer and centralized database utility. The purpose is to keep UI, business logic and persistence responsibilities separate.



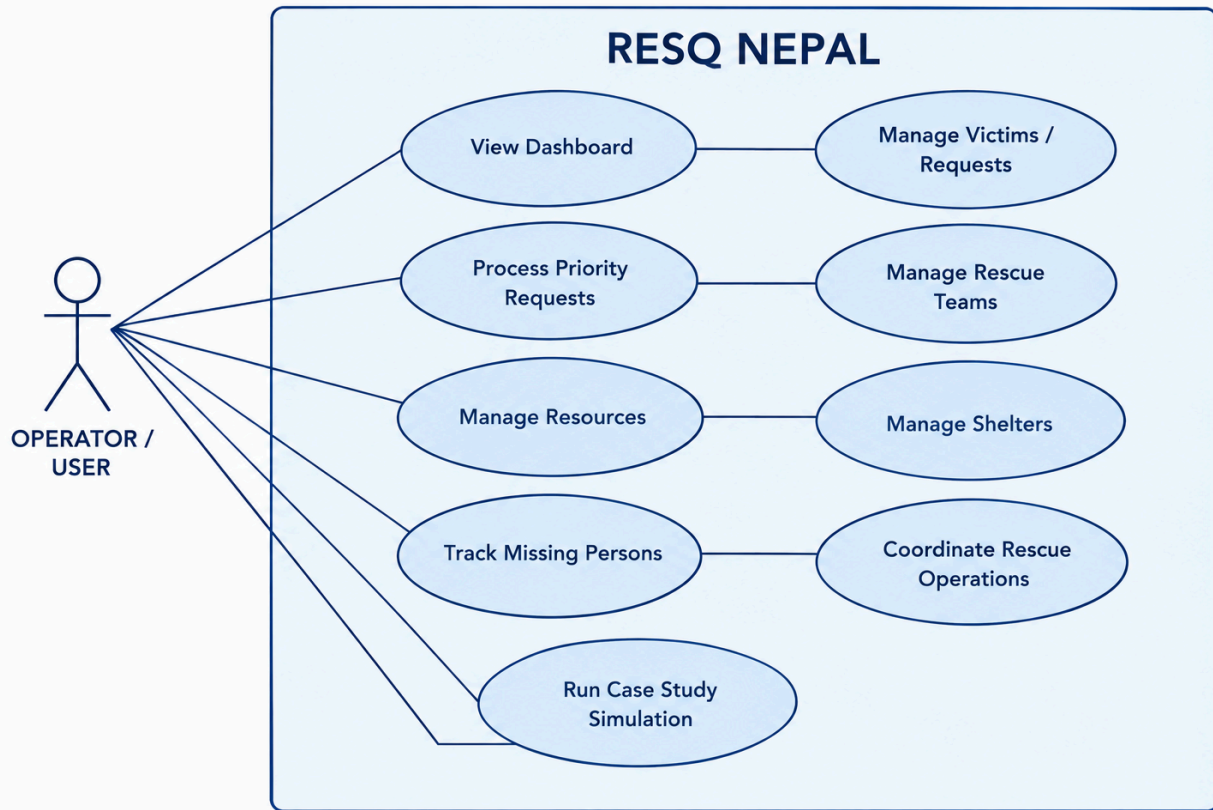
Architecture Components

- View:FXML layouts define JavaFX screens.
- Controller: Handles user events and updates the UI without direct SQL/core business logic.
- Service: Performs priority processing, team assignment, resource validation and capacity checks.
- Model: POJOs represent domain entities.
- Repository: DatabaseManager handles SQLite/JDBC operations using try-with-resources.
- Utility: DistanceUtil supports geographical distance calculations.

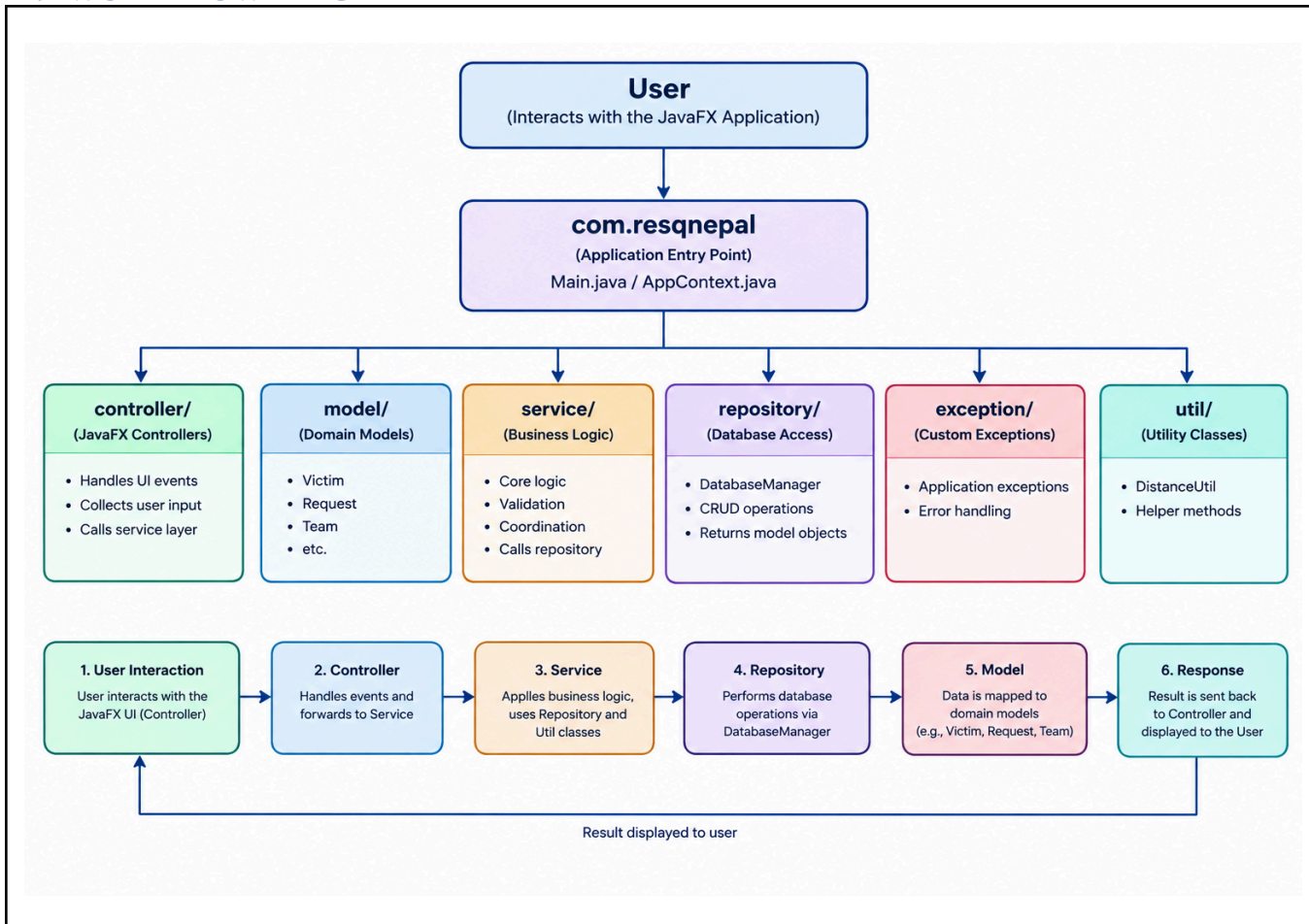
7. DESIGN DIAGRAMS

7.1 USE CASE DIAGRAM

The main actor is the Operator/User. The actor interacts with the application to manage emergencies and operational resources.



7.2 WORKFLOW DIAGRAM

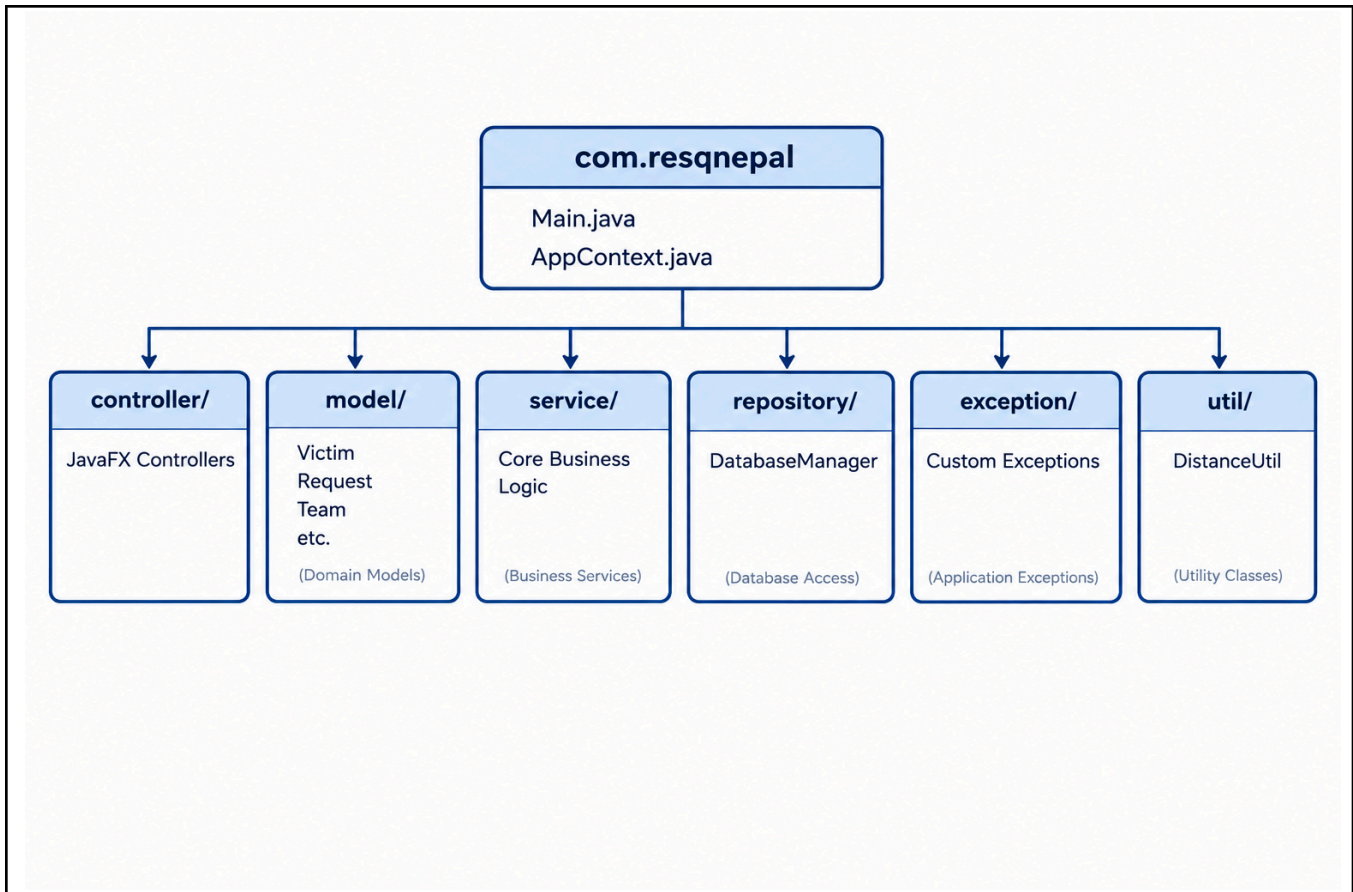


7.3 SEQUENCE DIAGRAM

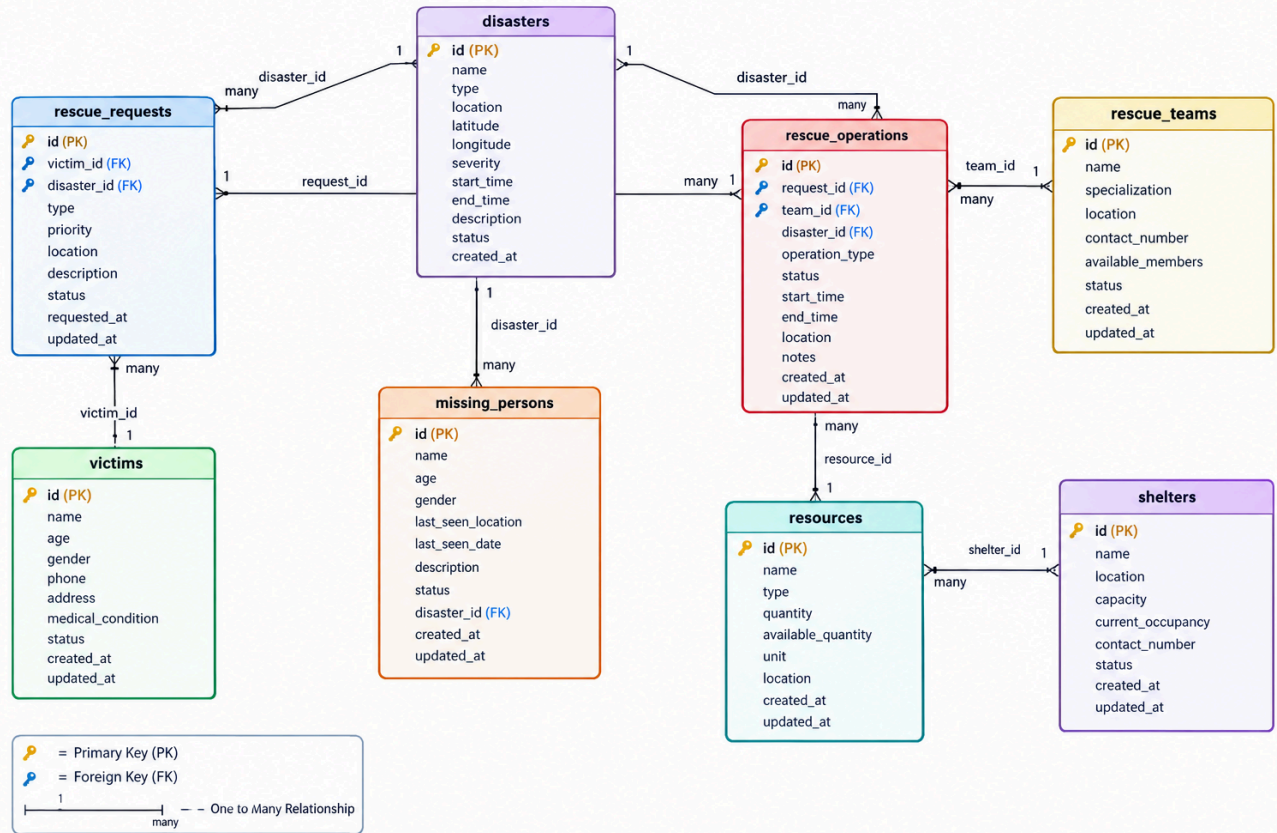
Representative interaction for processing a rescue request:

Participant	Interaction
Operator	Creates rescue request.
Controller	Receives input and validates it.
Service	Determines priority and manages the request.
PriorityQueue	Orders requests by severity.
Team Service	Filters available and specialized teams.
DistanceUtil	Calculates geographical distance.
Team Service	Selects nearest suitable team.
DatabaseManager	Stores operation and status changes.
Controller	Refreshes results for the operator.

7.4 CLASS / COMPONENT DIAGRAM



7.5 ER DIAGRAM



8. DESIGN DECISIONS & RATIONALS

Decision	Rationale
Java 17	Supports OOP, collections, exceptions, interfaces and the required application implementation.
JavaFX 17.0.2	Provides the desktop GUI and works with FXML-based views.
MVC	Separates presentation, event handling and business logic.
Service Layer	Keeps priority, team, resource and shelter rules outside controllers.
PriorityQueue + Comparator	Provides severity-based emergency-request processing.
Nearest Suitable Team	Combines availability, specialization and geographical proximity.
SQLite	Lightweight local relational storage for a desktop simulation.
DatabaseManager	Centralizes JDBC connection and database operations.
PreparedStatement	Separates SQL structure from supplied values and improves query safety.
Custom Exceptions	Represent domain-specific failures such as invalid requests or unavailable teams.
JUnit 5	Tests core business logic independently from the GUI.

9. IMPLEMENTATION

9.1 Project Structure

```
src/
├── main/java/com/resqnepal/
│   ├── Main.java
│   ├── AppContext.java
│   ├── controller/
│   ├── exception/
│   ├── model/
│   ├── repository/
│   ├── service/
│   └── util/
├── main/resources/com/resqnepal/ → FXML views
└── test/java/com/resqnepal/service/ → JUnit tests
```

9.2 Core Implementation

- Rescue requests are represented as model objects and placed into a PriorityQueue using a custom severity Comparator.
- Team assignment filters teams by availability and required specialization before comparing geographical distance.
- Resources are checked before allocation and can be released after use.
- Shelter capacity is checked before admitting evacuated individuals.
- DatabaseManager connects the application to SQLite using JDBC, try-with-resources and PRAGMA foreign_keys = ON.
- Custom exceptions include InvalidRequestException and NoTeamAvailableException.
- JUnit 5 is used for automated business-logic testing.

9.3 Technologies

Technology	Version / Role
Java	17
JavaFX	17.0.2
SQLite	Local database
sqlite-jdbc	JDBC driver
Maven	Build/dependency management
JUnit 5	Unit testing

9.4 Run & Test

Run: `mvn javafx:run`

Test: `mvn clean test`

10. SCREENSHOTS AND RESULTS

Figure 10.1: Dashboard and live-style statistics

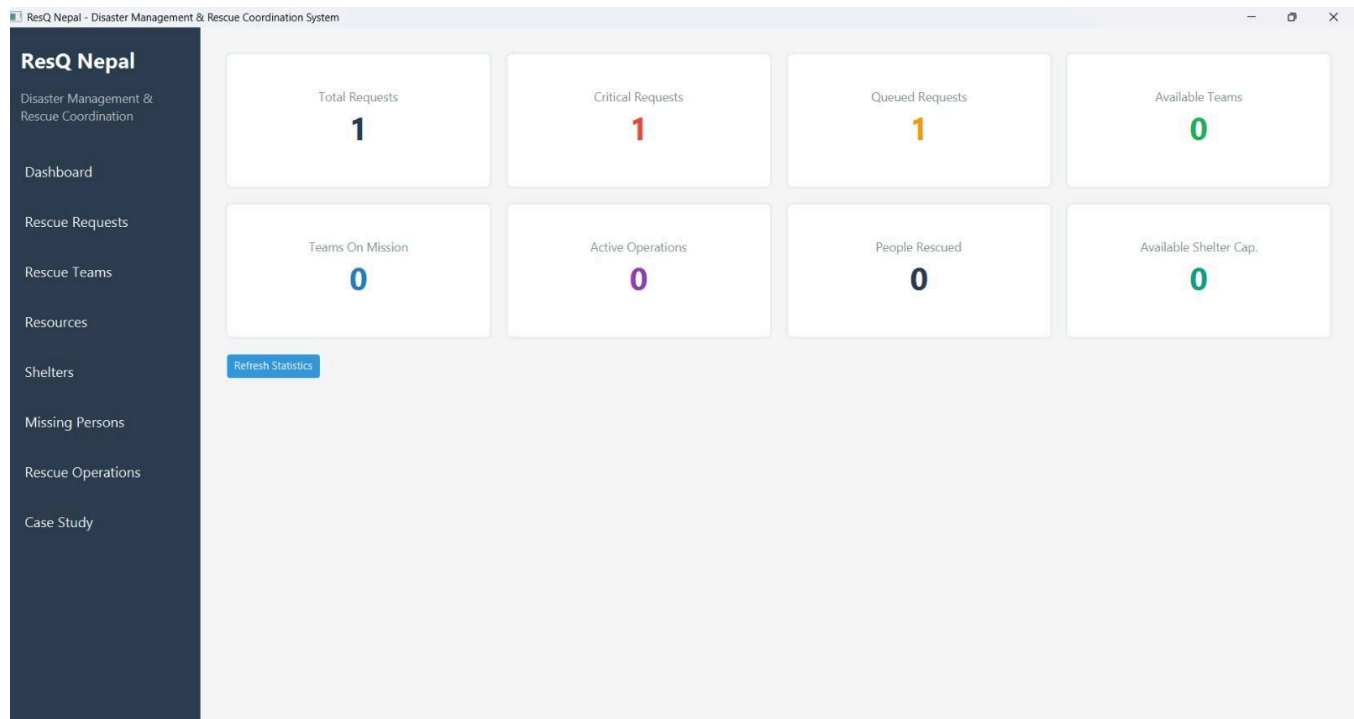


Figure 10.2: Rescue Request registration

The screenshot shows the 'Rescue Requests Management' form for registering a new request. The form includes the following fields:

- Requester Name: akash gupta
- Age: 34
- Phone Number: 1234567890
- Location: Rohini
- Address: A-123
- Type: flood
- Severity: 90
- Priority: CRITICAL (dropdown menu)
- Specialization: Doctor
- Description of the emergency: (empty text area)

A green 'Register Request' button is located at the bottom right of the form.

Figure 10.3: Priority-based request processing

The screenshot shows the 'Request Queue' table with a 'Process Next Request' button and a 'Refresh' button. The table contains the following data:

Request ID	Victim ID	Type	Severity	People Aff.	Specialization	Status	Description
REQ-1789632...	1000	Flood	CRITICAL	90	Doctor	QUEUED	

Figure 10.4: Affected System

Zone A

Simulation Scenario: Urban Flooding

Simulated Impact: High

Simulated Action: 3 Rescue Teams deployed for swift-water evacuation.

Zone B

Simulation Scenario: Landslide Blockage

Simulated Impact: Critical

Simulated Action: Engineering team assigned to clear debris. Medical airlift requested.

Zone C

Simulation Scenario: Stranded Village

Simulated Impact: Medium

Simulated Action: Resource airdrop in progress. 500 food rations allocated.

Zone D

Simulation Scenario: Evacuation Camp

Simulated Impact: Low

Simulated Action: Camp established safely. 200 simulation victims admitted to Shelter S-01.

Figure 10.5: Resource Management

Resources Management

Add New Resource

Doctor

Medical treatment

10

Delhi

rohini

AVAILABLE

Add Resource

Change Status Only

Inventory

Allocate Resource

Release Resource

Refresh List

Figure 10.6: Missing Persons

Missing Persons

Register Missing Person

MISSING

Register Person

Update Status

Records

Search

Refresh List

Record ID	Name	Last Known Location	Status
MP-1789632108450	Ivan	Delhi	MISSING

Figure 10.7: Shelter Management

Shelter Management

Add New Shelter

Location X

Location Y

☐ Medical Support

☐ Food Available

☐ Water Available

Max Capacity

Add Shelter

11. TESTING

The project uses automated JUnit 5 testing to verify important business rules.

Test Area	Expected Result
Request Queue	Higher-severity request is processed first.
Request Management	Invalid request is rejected/handled through validation or exception.
Missing Person	Register Missing Person
Shelter Capacity	Admission cannot exceed maximum capacity.

Request Queue

Process Next Request

Refresh

Request ID	Victim ID	Type	Severity	People Aff.	Specialization	Status	Description
REQ-1789632...	1000	Flood	CRITICAL	90	Doctor	QUEUED	

Rescue Requests Management

Register New Request

akash gupta		34	1234567890	
Rohini	A-123	flood		
90	CRITICAL	Doctor		
Description of the emergency				
<button>Register Request</button>				

Missing Persons

Register Missing Person

Full Name	Last Known Location	MISSING
<button>Register Person</button>	<button>Update Status</button>	

Records

SearchRefresh List

Record ID	Name	Last Known Location	Status
MP-1789632108450	Ivan	Delhi	MISSING

Shelter Management

Add New Shelter

Shelter Name		Max Capacity	
Location X	Location Y	☐ Medical Support	☐ Food Available ☐ Water Available
<button>Add Shelter</button>			

12. CHALLENGES FACED

- Deciding the priority of different emergency requests based on their severity.
- Managing priority along with team availability and their skills.
- Finding the nearest rescue team that is suitable for the incident.
- Keeping track of resources when they are given out or returned.
- Making sure shelters do not go over their capacity.
- Keeping JavaFX controllers separate from the main service logic.
- Managing the connections between different SQLite tables.
- Handling different errors using custom exceptions.
- Testing the main rules separately without depending on the user interface.

13. LEARNING & KEY TAKEAWAYS

- Applied OOP, collections, abstraction, interfaces and exception handling.
- Learned JavaFX/FXML and MVC-based application organization.
- Applied PriorityQueue and Comparator to a realistic problem.
- Practiced JDBC, SQL, SQLite and PreparedStatements.
- Understood service-layer separation and database integrity.
- Learned how unit testing can validate business rules.
- Improved system-design and documentation skills.

14. FUTURE ENHANCEMENTS

- GPS and map-based visualization of victims, teams and incidents.
- Real-time notifications for operators and rescue teams.
- Mobile application support for field responders.
- Cloud synchronization for multi-device coordination.
- Role-based authentication and stronger security.
- Integration with weather, earthquake, flood and other disaster APIs.
- AI/ML-based severity classification and demand prediction.
- Advanced resource optimization across simultaneous disasters.
- Analytics for response time, resource utilization and shelter occupancy.

15. CONCLUSION

ResQ Nepal shows how different Java concepts can be used together for disaster-response coordination. The PriorityQueue helps process requests based on severity, while team assignment checks availability, specialization and distance.

The MVC structure keeps the JavaFX interface separate from controllers and service logic. SQLite and JDBC are used for local data storage, while custom exceptions and JUnit 5 help with error handling and testing.

The project is an educational simulation, not a real emergency system. Its modular design can be improved later with mapping, real-time communication, mobile apps, cloud storage and analytics.

16. REFERENCES

10. ResQ Nepal — GitHub Repository, singhikhushi30-crypto , <https://github.com/singhikhushi30-crypto/ResQ-Nepal-Khushi>
11. Oracle Java Documentation — <https://docs.oracle.com/en/java/>
12. OpenJFX Documentation — <https://openjfx.io/>
13. SQLite Documentation — <https://www.sqlite.org/docs.html>
14. JUnit 5 User Guide — <https://junit.org/junit5/docs/current/user-guide/>
15. Apache Maven Documentation — <https://maven.apache.org/guides/>

Project Disclaimer

This application is an educational simulation. The data displayed is fictional simulation data and does not represent official Nepal statistics or an actual deployment.